Module bibliography — renderer & physics

Reading list mapped concept-by-concept to the code in `lib/renderer.{h,c}` and `lib/physics.{h,c}`. For the broader project reading list, see `../../REFERENCES.md`.

Conventions: - **RTR4** = *Real-Time Rendering*, 4th ed. (Akenine-Möller et al.) - **RTCD** = *Real-Time Collision Detection* (Christer Ericson) - **GEA** = *Game Engine Architecture*, 3rd ed. (Jason Gregory) - **GPP** = *Game Programming Patterns* (Robert Nystrom) - **FGED1/2** = *Foundations of Game Engine Development* Vol. 1 (Math) / Vol. 2 (Rendering) (Lengyel)

---

renderer

Renderable pool, draw list, frame stats

- GPP — *Object Pool*, *Data Locality*. Free-list vs active-flag tradeoff; why a fixed array of `active` slots is fine here.
- GEA §11 (Rendering Engine). How draw queues / render lists are structured in shipping engines.

Tick→frame interpolation (`alpha`, `curr`/`prev` transforms)

- Glenn Fiedler — *Fix Your Timestep!* (gafferongames.com). Justifies the `pre-update copy → interpolate on render` pattern verbatim.
- GEA §8 (The Game Loop). Why visual update is decoupled from logic update.

Element-wise matrix lerp vs decompose / slerp

- *3D Math Primer for Graphics and Game Development* — Dunn. Quaternion and matrix chapters.
- FGED1 §2-§3 (Transforms). Explicit blend of matrices vs TRS decomposition.
- Jonathan Blow — *Understanding Slerp, Then Not Using It* (archived gamasutra post). When normalized lerp or element-wise is good enough.

Frustum culling (Gribb–Hartmann)

- Gribb & Hartmann — *Fast Extraction of Viewing Frustum Planes from the World-View-Projection Matrix*. The paper the implementation follows.
- RTR4 §22 (Intersection Test Methods), §19 (Acceleration Algorithms). Plane-sphere, plane-AABB, hierarchical culling.
- FGED2 §9 (Visibility and Occlusion). Same engineering angle as the code.

Bounding spheres (combined AABB → sphere; max-axis-scale on transform)

- RTCD §4 (Bounding Volumes). AABB→sphere "center + farthest corner" with looseness discussion; Ritter and Welzl alternatives.
- *3D Math Primer* — Dunn. Bounding volumes for the intuitive view.

Material sorting (group by `materialID`, front-to-back inside)

- Christer Ericson — *Order your graphics draw calls around!* (realtimecollisiondetection.net/blog). Canonical short post on sort-key construction.
- RTR4 §18 (Pipeline Optimization). Why `material → depth` minimizes state changes and helps early-Z.
- FGED2 §10-§11. Material systems and submission.

Early-Z / why front-to-back helps

- RTR4 §23 (Graphics Hardware). Early-Z, hierarchical-Z.
- Learn OpenGL — *Advanced OpenGL → Depth testing*. Short and practical.

Hessian normal form (the `FrustumPlane` struct)

- *3D Math Primer* Appendix A (Geometric primitives) or FGED1 §3 (Geometry). 15-minute reads.

Minimal reading order for renderer

1. Fiedler, *Fix Your Timestep!* — `curr/prev` and `alpha`.
2. Gribb–Hartmann paper — frustum culling as implemented.
3. Ericson, *Order your graphics draw calls around!* — `(materialID, distSq)` sort key.
4. RTR4 §18 + §22 — pipeline context and tests.
5. RTCD §4 — bounding volumes with rigor.

---

physics

Module design / character physics

- *Game Physics Engine Development*, 2nd ed. — Millington (early chapters). Pure tests vs stateful world; kinematic mover without full rigid bodies.
- GPP — *Object Pool*. Same fit as the renderer pool, applied to `Collider[MAX_COLLIDERS]`.
- GEA §13 (Collision and Rigid Body Dynamics). Panoramic view of what is and isn't implemented here.

Primitives: AABB, sphere, capsule

- RTCD §4 (Bounding Volumes). Canonical definitions, tightness vs cost. Capsule as a swept sphere along a segment.
- *3D Math Primer* §A.13 (Geometric Primitives) for the intuitive view.

Intersection tests (the six `Test*` functions)

All from **RTCD §5 (Basic Primitive Tests)**: - **Sphere vs AABB** — §5.1.3 (closest-point-on-AABB → distance to center). - **Sphere vs Capsule** — §5.1.7 (point-segment distance + sum of radii). - **Capsule vs Capsule** — §5.1.9 (segment-segment distance — the non-trivial one; cross-reference Eberly `DistSegSeg`). - **Capsule vs AABB** — combine segment-vs-AABB with the box expanded by `radius`. No direct test; build

from §5.1. - **Ray vs AABB** — §5.3.3 (Kay-Kajiya slab test). - **Ray vs Capsule** — §5.3.7 (infinite cylinder + two end spheres).

Auxiliary outside RTCD: **David Eberly — *Geometric Tools*** (geometrictools.com). Per-test PDFs (`DistanceSegmentSegment`, `IntersectionRayCapsule`) with near-translatable C++.

Layer / mask filtering (`(A.mask & B.layer) && (B.mask & A.layer)`)

- GEA §13.3 (Collision Filtering). Justifies the symmetric double check.
- **Bullet** and **PhysX** docs on collision groups. Same pattern in industry, online.

Broad phase (current O(n^2) with stats; what to upgrade to)

- RTCD §7 (Spatial Partitioning). Uniform grid, sweep-and-prune, BVH — what's *not* implemented yet, for when `MAX_COLLIDERS` grows.
- RTCD §6 (Bounding Volume Hierarchies) if scaling further.

MoveAndCollide iterative resolution (normal × penetration loop)

- RTCD §5.5 (Continuous Collision Detection). Read to understand why CCD isn't needed yet (slow objects relative to size).
- Erin Catto — GDC slides at box2d.org/publications. Especially *Modeling and Solving Constraints* and *Fast and Simple Physics using Sequential Impulses*. Same pattern: resolve per iteration, normal × penetration.
- Glenn Fiedler — *Integration Basics*, *Collision Response* (gafferongames.com). Short web version of the same material.

Contact normal convention ("from B toward A")

- RTCD §1.5 (A Math Review). Ericson uses this convention throughout, so copying it keeps the code aligned with the book's formulas.

Grounded detection (`normal.y > 0.7`)

- *Game Programming Gems 4 — Simple Character Physics*.
- Quake / Source engine character controller writeups. The `normal.y > 0.7` (slope < ~45°) threshold originates there.
- Bungie GDC talks on Halo character physics (public slides). Sweep-and-step and the ground threshold are explained.

Raycast API design (closest hit, ignore-self)

- RTCD §5.3 (covered above).
- Godot `PhysicsDirectSpaceState` and Unity `Physics.Raycast` API docs. Reading both helps decide which overloads earn their keep.

Overlap sphere (AOE queries)

- RTCD §5.1.4 (sphere-AABB), §5.1.5 (sphere-sphere). Nothing beyond that for the bounded `outHandles[maxResults]` loop.

Trigger detection (overlap without resolution)

- GEA §13.5 (Collision Filtering and Triggers). Collider/trigger split and why triggers run in a separate pass (what `PhysicsUpdate` does).

Minimal reading order for physics

1. RTCD §4 + §5 — covers ~80% of the implementation.
2. Erin Catto — *Fast and Simple Physics using Sequential Impulses* (GDC slides). Why the iterative resolver converges.
3. Gaffer on Games — *Collision Response*. Short and applied.
4. Eberly — `DistanceSegmentSegment.pdf`. The only non-trivial test in the module.
5. GEA §13. Where to look when CCD, joints, or a spatial broad phase become relevant.